

CSE574: Cloud Computing

Indian Institute of Technology (BHU) Varanasi

Assignment 1: Deploying a Web Application on AWS EC2

Instructor: Dr. Prasenjit Chanak

Name: Jayesh Krishan Puri
Branch: Mining Engineering, 3rd Year B.Tech
Roll No: 24155058
Date: 29 August 2026
GitHub <https://github.com/Jayesh-Dev21>

1. Cloud Platform and VM Setup

An AWS account was created and an **EC2 instance** was launched on **Amazon Web Services (AWS)** to serve as the cloud VM for deploying the web application. The instance was configured with Ubuntu Server and accessed via SSH using a key pair.

Cloud Provider	Amazon Web Services (AWS)
Service	EC2 (Elastic Compute Cloud)
OS (Guest)	Ubuntu Server (latest LTS)
Instance Type	t2.micro (Free Tier eligible)
Region	ap-south-1 (Mumbai)
Key Pair	pair.pem (SSH authentication)
Security Group	SSH (22), HTTP (80), HTTPS (443)

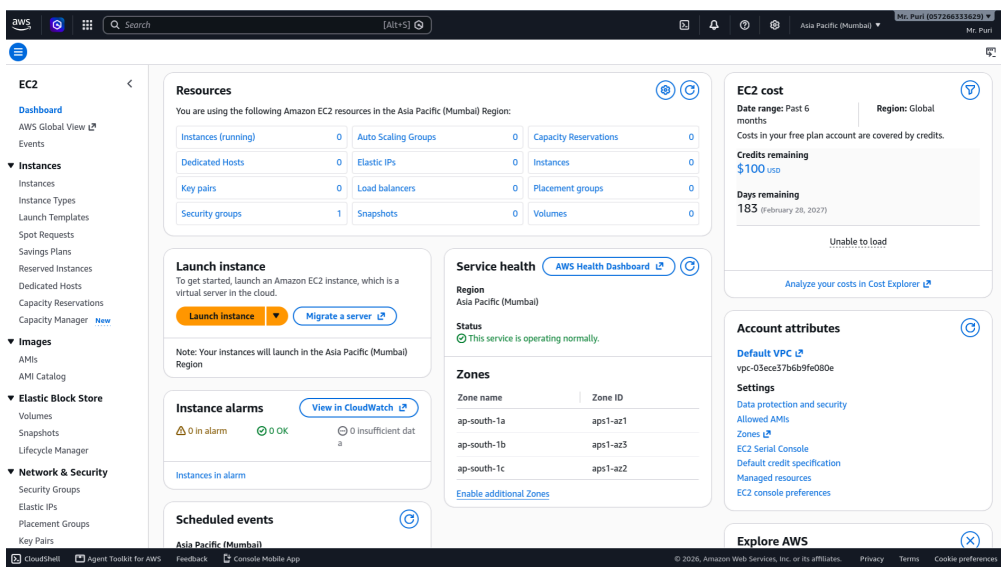


Figure 1: AWS Management Console – EC2 dashboard

2. SSH and Instance Configuration

2.0.1. Step 1: Key Pair and SSH Setup

A key pair (.pem file) was downloaded during instance creation. SSH was configured to connect to the EC2 instance using the private key.

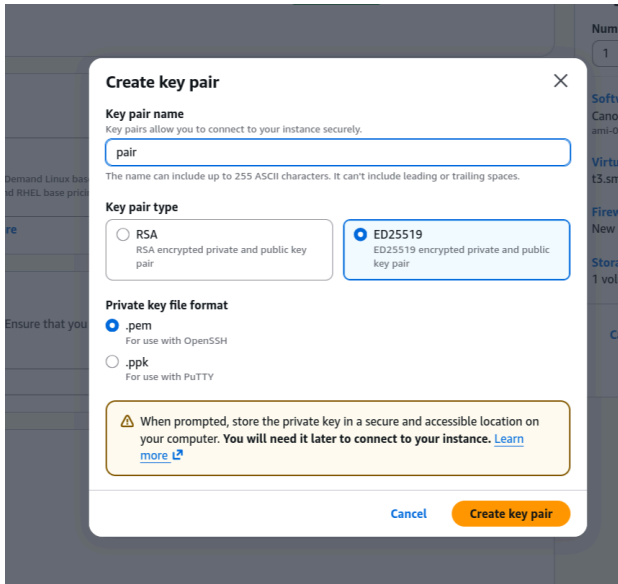


Figure 2: Key pair download

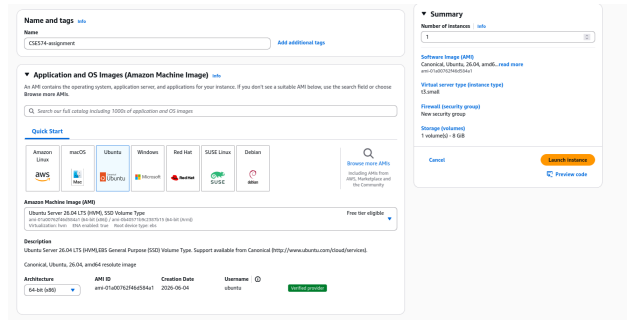


Figure 3: Instance launch configuration

2.0.2. Step 2: SSH Connection and Verification

The EC2 instance was accessed via SSH from the local terminal using the downloaded key pair. The instance state confirmed it was running successfully.



Figure 4: Instance state – running

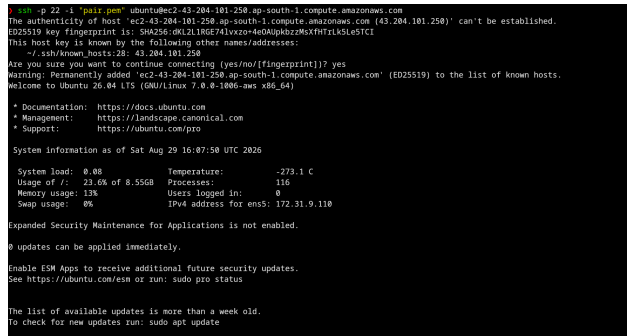


Figure 5: SSH connection established

2.0.3. Step 3: System Update and Package Installation

After connecting, the system packages were updated and essential software was installed including **nginx**, **curl**, **git**, **Python3**, and the **AWS CLI**.

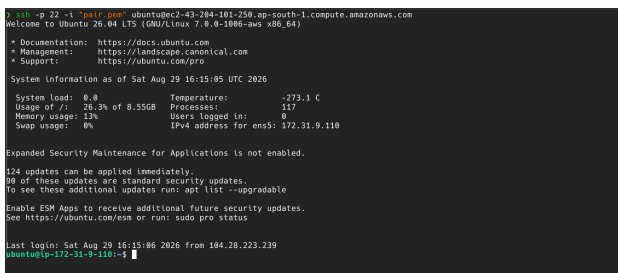


Figure 6: System update and package install

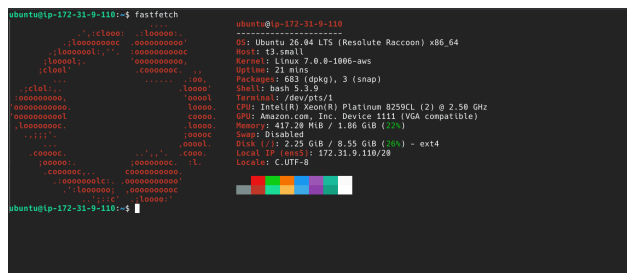


Figure 7: Nginx, curl, git, AWS CLI installed

3. Deploying the Flask Application

3.0.1. Step 4: Transferring Code to the EC2 Instance

The Flask application code was transferred from the local machine to the EC2 instance using **scp** (secure copy).

```

scp -i "pair.pem" -r app.py requirements.txt templates/ ubuntu@ec2-43-204-101-250.ap-south-1.compute.amazonaws.com:~/assignment1
app.py 100% 944 20.3KB/s 00:00
requirements.txt 100% 80 8.6KB/s 00:00
working.html 100% 3752 88.4KB/s 00:00
index.html 100% 3637 88.6KB/s 00:00

```

Figure 8: scp transfer of application code

```

ubuntu@ip-172-31-9-110:~$ ls
assignment1
ubuntu@ip-172-31-9-110:~$ ls assignment1/
app.py  requirements.txt  templates
ubuntu@ip-172-31-9-110:~$

```

Figure 9: Files received on EC2

3.0.2. Step 5: Flask App Deployment

The Flask application was deployed on the EC2 instance using **Gunicorn** as the WSGI server. The app was configured to listen on `0.0.0.0:5000` and was managed using a **systemd** service for automatic restarts.

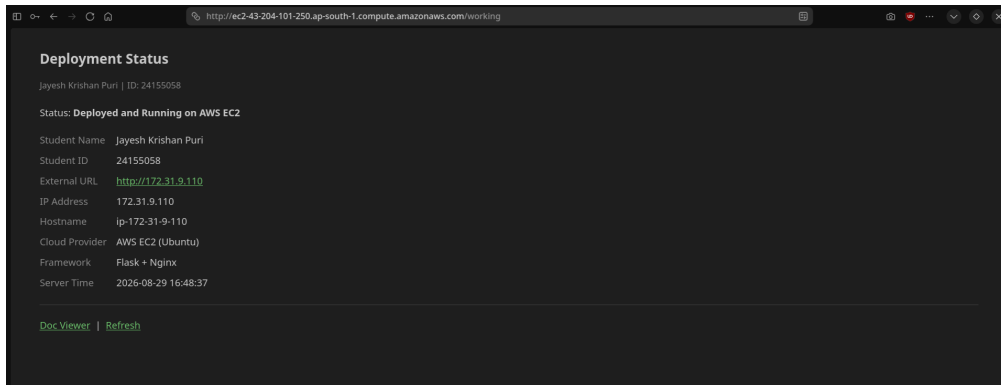


Figure 10: Flask application deployed on EC2

3.0.3. Step 6: Running Application

The Flask application was started and verified to be running correctly on the EC2 instance.

```

ubuntu@ip-172-31-9-110:~$ cd assignment1/
ubuntu@ip-172-31-9-110:~/assignment1$ ./deploy.sh
./deploy.sh: line 1: ./deploy.sh: Permission denied
ubuntu@ip-172-31-9-110:~/assignment1$ ls -l
total 12
-rw-r--r-- 1 ubuntu ubuntu 1024 Aug 29 16:48 app.py
-rw-r--r-- 1 ubuntu ubuntu 80 Aug 29 16:48 requirements.txt
-rw-r--r-- 1 ubuntu ubuntu 3752 Aug 29 16:48 working.html
-rw-r--r-- 1 ubuntu ubuntu 3637 Aug 29 16:48 index.html
ubuntu@ip-172-31-9-110:~/assignment1$ python3 -m http.server 5000
Python 3.10.12 shell
Serving HTTP on 0.0.0.0 port 5000 (http://0.0.0.0:5000/)...
127.0.0.1:5000 - "GET / HTTP/1.1" 200 -

```

Figure 11: Flask app running on port 5000

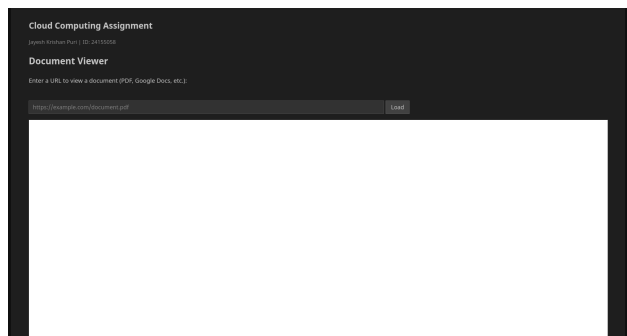


Figure 12: Document viewer page loaded

4. Application Endpoints

The web application provides two main endpoints:

- `/` – A document viewer page with a URL input field to view documents (PDFs, Google Docs, etc.) directly in the browser, along with a “Download to S3” button.
- `/working` – A deployment status page displaying the student details, external URL, IP address, hostname, cloud provider, framework, and server timestamp.

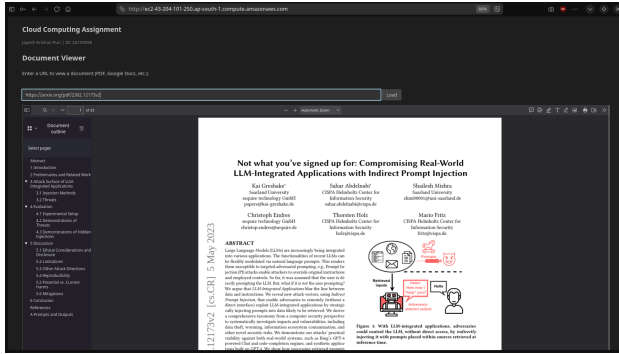


Figure 13: Document loaded in viewer

```
python3-pip is already the newest version (25.1.1+dfsg-1ubuntu2).
python3-venv is already the newest version (3.14.3-0ubuntu2).
nginx is already the newest version (1.28.3-2ubuntu1.10).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 110
Requirement already satisfied: Flask==3.1.1 in ./venv/lib/python3.14/site-packages (fr
Requirement already satisfied: gunicorn==23.0.0 in ./venv/lib/python3.14/site-packages (
Requirement already satisfied: blinker>=1.9.0 in ./venv/lib/python3.14/site-packages (
Requirement already satisfied: click>=8.1.3 in ./venv/lib/python3.14/site-packages (fr
Requirement already satisfied: itsdangerous>=2.2.0 in ./venv/lib/python3.14/site-packa
Requirement already satisfied: Jinja2>=3.1.2 in ./venv/lib/python3.14/site-packages (f
Requirement already satisfied: MarkupSafe>=2.1.1 in ./venv/lib/python3.14/site-package
Requirement already satisfied: Werkzeug>=3.1.0 in ./venv/lib/python3.14/site-packages (f
ubuntu@ip-172-31-9-110:~/assignment1$ ./stop.sh
Stopping Flask app...
Stopping nginx...
Done. Both services stopped.
ubuntu@ip-172-31-9-110:~/assignment1$ ./deploy.sh
== Cloud Computing Assignment - Deployment ==
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
110 packages can be upgraded. Run 'apt list --upgradable' to see them.
python3 is already the newest version (3.14.3-0ubuntu2).
python3-pip is already the newest version (25.1.1+dfsg-1ubuntu2).
python3-venv is already the newest version (3.14.3-0ubuntu2).
nginx is already the newest version (1.28.3-2ubuntu1.10).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 110
Requirement already satisfied: Flask==3.1.1 in ./venv/lib/python3.14/site-packages (fr
Requirement already satisfied: gunicorn==23.0.0 in ./venv/lib/python3.14/site-packages (
Requirement already satisfied: blinker>=1.9.0 in ./venv/lib/python3.14/site-packages (
Requirement already satisfied: click>=8.1.3 in ./venv/lib/python3.14/site-packages (fr
Requirement already satisfied: itsdangerous>=2.2.0 in ./venv/lib/python3.14/site-packa
Requirement already satisfied: Jinja2>=3.1.2 in ./venv/lib/python3.14/site-packages (f
```

Figure 14: Deploy and stop scripts working

5. Nginx Configuration

Nginx was configured as a reverse proxy to forward requests from port 80 to the Gunicorn server running on port 5000. The configuration was placed in `/etc/nginx/sites-available/flask_app` and symlinked to `sites-enabled`.

Framework	Flask (Python)
WSGI Server	Gunicorn (3 workers)
Web Server	Nginx (reverse proxy, port 80)
App Port	5000
Process Manager	systemd (flask_app.service)

6. AWS S3 Integration

6.0.1. Step 7: S3 Bucket Creation

An S3 bucket named `docustore-cse574` was created in the `ap-south-1` (Mumbai) region for storing documents. An S3 Access Point was also configured for the bucket.

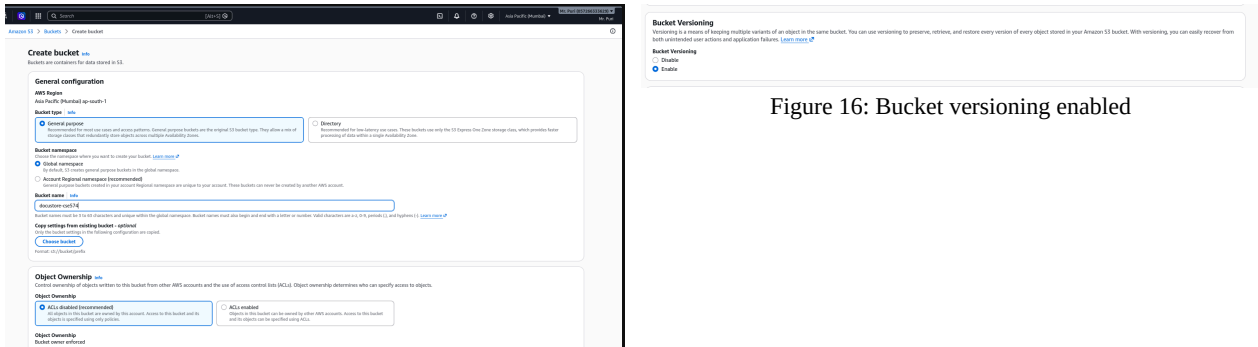


Figure 15: S3 bucket created

6.0.2. Step 8: S3 Upload via boto3 SDK

The application uses **boto3**, the AWS SDK for Python, to upload documents to S3. When a user enters a document URL and clicks “Download to S3”, the application fetches the document and uploads it to the bucket using `put_object`.

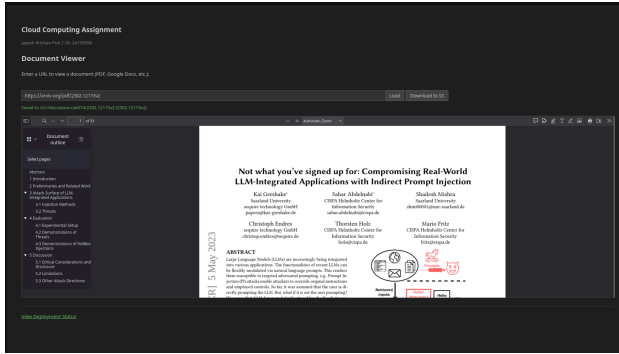


Figure 17: Upload via SDK – any PDF from URL

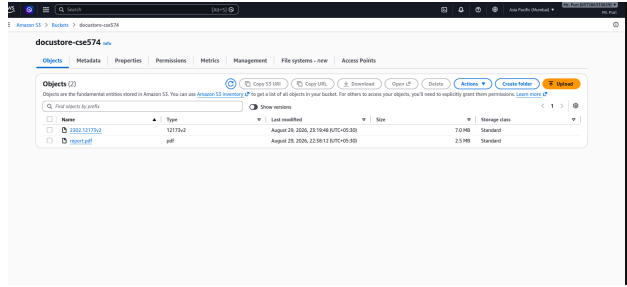


Figure 18: All files visible in S3 bucket

6.0.3. Step 9: Cross-Verification

The uploaded documents were verified by viewing them from the S3 bucket through the application's document viewer, confirming end-to-end functionality.

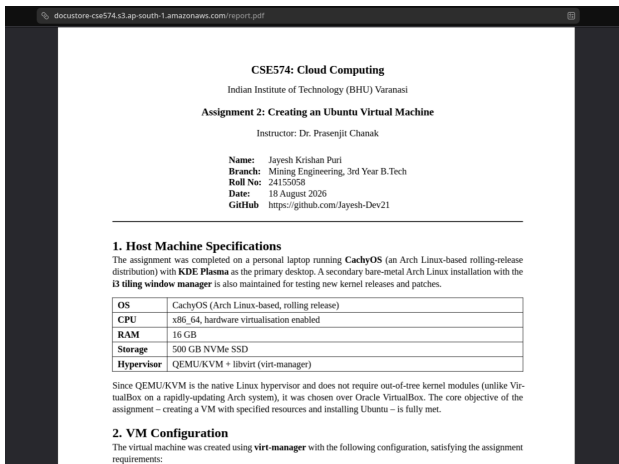


Figure 19: S3 stored document viewed in app

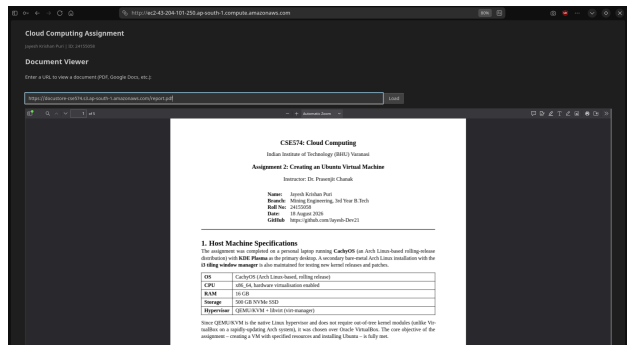


Figure 20: Assignment 2 report viewed on Assignment 1 app

7. Challenges Faced

The primary challenge was **AWS credential configuration**. The initial deployment encountered an `Unable to locate credentials` error, which was resolved by running `aws configure` on the EC2 instance. Subsequently, an `AccessDenied` error appeared because the IAM user lacked `s3:PutObject` permissions on the target bucket. This was fixed by attaching the `AmazonS3FullAccess` policy to the IAM user through the AWS IAM console.

8. Deployment Verification

The application was verified by accessing both endpoints on the EC2 public IP:

- `http://<EC2_IP>/` – The document viewer page loaded successfully with the URL input field and Load / Download to S3 buttons.
- `http://<EC2_IP>/working` – The deployment status page displayed the correct student details (Jayesh Krishan Puri, 24155058), external URL, IP address, hostname, and server timestamp.

The Flask application runs under **systemd** (`flask_app.service`) with Gunicorn serving 3 worker processes, and Nginx proxying traffic on port 80.

9. Service URLs and Current Status

App URL	<code>http://<EC2_IP>/</code> (now stopped)
Status Page	<code>http://<EC2_IP>/working</code> (now stopped)
S3 Bucket	<code>s3://docustore-cse574</code>
S3 Access Point	<code>assignment-suyan8hc8ibfazmowpudzcio5uhwnaps3a-s3alias</code>
S3 Region	ap-south-1 (Mumbai)

Note: The EC2 instance and Flask service have been **stopped** after successful verification to avoid unnecessary compute charges on the AWS Free Tier. The S3 bucket docustore-cse574 remains active and all uploaded documents are intact.

10. Source Code

The complete source code for this assignment is available at:

<https://github.com/Jayesh-Dev21/cse574>

CSE574: Cloud Computing | IIT (BHU) Varanasi | 29 August 2026